ERBRAINS IT SOLUTIONS

Senior Mobile Developer

Take-Home Technical Assignment

Wearable Health & Shopping Mobile App

Stack: Flutter · Node.js · PostgreSQL

Time limit: 72 hours

Overview

Build a production-style Flutter mobile application (Android and iOS) that simulates a wearable health device and includes a basic shopping flow. The assignment is designed to demonstrate engineering ability and architectural thinking, not production-level UI polish.

The goal is to evaluate your ability to design and build a complete mobile solution across:

- Cross-platform mobile development
- Wearable / device integration architecture
- Offline data handling and synchronisation
- Backend and API integration
- Database design
- Authentication
- E-commerce functionality
- Android and iOS considerations

---

1. Mobile Application

Build the application using **Flutter**, supporting both Android and iOS.

Authentication

- Login
- Logout
- Basic authenticated session

You may use mock authentication or your own backend implementation.

Dashboard

After login, display the following live values:

- Heart Rate
- SpO_2
- Steps
- Device Battery
- Device Connection Status

```
Heart Rate      78 BPM
SpO2            98%
Steps           6,420
Battery         72%
Status          Connected
```

Code Quality & Architecture

The solution should follow a clean, maintainable and scalable project structure. We will evaluate separation of concerns, reusable components, error handling, testability, and overall code quality. Avoid unnecessary complexity; prioritize clear and maintainable engineering decisions.

2. Wearable Simulation

You will **not** receive a physical wearable device or vendor SDK. Instead, create a mock wearable / device layer.

The simulated device should:

- Connect
- Disconnect
- Reconnect
- Provide battery information
- Generate heart-rate readings
- Generate SpO_2 readings
- Generate step readings

Generate new health readings periodically (for example, every few seconds). Example reading:

```
{
  "deviceId": "FITRING-001",
  "heartRate": 78,
  "spo2": 98,
  "steps": 6420,
  "timestamp": "2026-08-17T10:30:00Z"
}
```

---

3. Architecture Requirement

Structure the application so that the mock wearable can later be replaced by a real vendor SDK. Your architecture should clearly separate the layers:

```
Flutter Application
        |
Wearable Service / Interface
        |
Mock Wearable Implementation
```

In your README, explain how you would replace the mock implementation with a real SDK:

```
Flutter
   |
Native Bridge
   |
Android SDK       iOS SDK
   |                 |
Kotlin/Java       Swift/Obj-C
   |                 |
        Smart Ring
```

For example, explain whether you would use:

- Flutter Platform Channels
- Native plugins
- Another architecture (and why)

---

4. Connection Handling

The application should handle:

- Device connected
- Device disconnected
- Automatic reconnect attempt
- Connection failure
- User manually reconnecting

Document your retry/reconnection strategy in the README.

---

5. Local Health Data

Store health readings locally on the device. The user should be able to open a History screen and see previous readings, including:

- Daily history
- Weekly summary
- Heart-rate chart
- SpO_2 chart
- Steps summary

Do not load unlimited raw records into the UI at once. Explain how your application would behave if the device generated a very large number of readings.

---

6. Offline Synchronisation

The application must continue collecting and storing data when the internet is unavailable. When connectivity returns:

```
Local Data
   |
Sync Queue
   |
Backend API
```

Implement or clearly demonstrate:

- Offline storage
- Pending sync

- Retry after failure
- Duplicate prevention

Target scenario: The device generates 100 readings while the phone has no internet. When internet access returns, the app should sync those readings to the backend.

---

7. Backend

Build a backend API. Preferred stack: **Node.js + NestJS/Express + PostgreSQL**. You may use another suitable backend technology if you explain your choice.

Minimum APIs:

```
POST /auth/login

POST /devices
GET  /devices

POST /health/readings
GET  /health/readings
GET  /health/summary

GET  /products
GET  /products/:id

POST /cart
GET  /cart

POST /orders
GET  /orders
```

---

8. Database

Create a proper relational database structure. At minimum, consider these tables:

```
users
devices
health_readings
products
cart_items
orders
order_items
```

Provide your database schema / ER diagram or equivalent explanation. We want to see how you handle relationships between users, devices, health data and orders.

---

9. Shopping Module

Build a basic shopping flow:

```
Products  ->  Product Details  ->  Add to Cart
   ->  Cart  ->  Place Order  ->  Order History
```

You do not need to implement a real payment gateway. Focus on: product listing, product details, cart, quantity, order creation, and order history.

---

10. Error Handling

Consider and handle situations such as:

- Bluetooth / device disconnect
- API failure
- No internet
- Backend unavailable
- Duplicate health readings
- Authentication failure
- Failed synchronisation

Explain your approach in the README.

11. Testing

Include meaningful automated tests for important application logic. At minimum, test critical areas such as health-data processing, synchronization/duplicate prevention, or cart/order logic.

100% test coverage is not required. Focus on testing the parts where incorrect behavior could cause data loss, inconsistent state, or incorrect business results.

---

12. Deliverables

1. GitHub repository
2. Flutter application source code
3. Backend source code
4. Database schema
5. Android APK
6. iOS build / TestFlight (preferred where the candidate has access to the required Apple development environment)

A README containing:

- Architecture
- Setup instructions
- API documentation
- Database design
- Wearable integration approach

- Offline synchronization approach
- Major technical decisions and trade-offs

13. Technical Walkthrough

After submission, we will hold a 15–20 minute technical walkthrough. Be prepared to explain.

Time Limit

72 hours. This assignment is intended to demonstrate engineering ability, not production-level UI polish.

Scope Guidance

Do not spend excessive time on advanced animations, visual effects, or production-level UI polish. The evaluation will focus primarily on architecture, functionality, reliability, data handling, backend integration, wearable integration approach, and technical decision-making.